

I will provide system requirements. Create the system using these requirements. 1. The system shall support tickets for a single ride 2. The system shall support tickets for a set period 3. The system shall support tickets bought in bundles 4. The system shall allow tickets to be bought on a mobile application 5. The system shall allow tickets to be bought on a web application 6. The system shall adjust the ticket price based on who the user identifies as (student, senior, regular) 7. The system shall adjust the ticket price based on the ticket price (single, period, bundle) 8. The system shall support discounts for students 9. The system shall support offers for seniors 10. The system shall allow users to purchase multiple tickets at once 11. The system shall handle invalid ticket orders (e.g., selecting a ticket without specifying a valid quantity) 12. The system shall be reliably calculate ticket prices 13. The system shall keep users personal information private 14. The system shall keep users payment information private



Got it — you're essentially describing a **ticketing system for public transport (or similar)** with pricing logic, multiple channels, and privacy constraints. I'll turn your requirements into a clear **system design + structure** you can implement.

1. High-Level Architecture

Core Components

- **Client Applications**
 - Mobile App
 - Web App
- **Backend Services**
 - Ticket Service
 - Pricing Service
 - Order Service
 - User Service
 - Payment Service
- **Database**

- Users
- Tickets
- Orders
- Pricing rules

2. Domain Model

Entities

User

```
User
- id
- name
- email
- userType (STUDENT, SENIOR, REGULAR)
```



Ticket

```
Ticket
- id
- type (SINGLE, PERIOD, BUNDLE)
- basePrice
```



Order

```
Order
- id
- userId
- tickets[] (TicketOrderItem)
- totalPrice
- status
```



TicketOrderItem

```
TicketOrderItem
- ticketId
- quantity
- calculatedPrice
```



3. Core Services

3.1 Ticket Service

Handles ticket definitions.

Functions:

- `getAvailableTickets()`
- `getTicketById()`

3.2 Pricing Service (⚡ critical logic)

Responsible for:

- Requirement 6, 7, 8, 9, 12

Pricing Rules

Base Formula

$$\text{Final Price} = \text{Base Price} \times \text{User Modifier} \times \text{Ticket Modifier} \times \text{Quantity}$$



Example Modifiers

Category	Modifier
Student	0.8 (20% discount)
Senior	0.85 (15% offer)
Regular	1.0
Ticket Type	Modifier
Single	1.0
Period	0.9
Bundle	0.75

Pricing Function (Pseudo-code)

</> Python



▶ Kör

```
def calculate_price(user, ticket, quantity):  
    if quantity <= 0:  
        raise InvalidOrderException("Quantity must be greater than 0")  
  
    user_modifier = get_user_modifier(user.userType)  
    ticket_modifier = get_ticket_modifier(ticket.type)  
  
    price = ticket.basePrice * user_modifier * ticket_modifier * quantity  
  
    return round(price, 2)
```

3.3 Order Service

Handles:

- Requirement 10, 11

Responsibilities

- Validate order
- Calculate total price
- Store order

Validation Rules

- Quantity must be > 0
- Ticket must exist
- User must exist

Order Flow

User selects tickets →
Frontend sends request →
Order Service validates →
Pricing Service calculates →
Payment Service processes →
Order saved →
Tickets issued



3.4 Payment Service

Handles:

- Requirement 14 (privacy)

Key Design Choices

- NEVER store raw payment data
 - Use third-party providers (Stripe, Adyen, etc.)
 - Store only:
 - transaction ID
 - masked card info
-

3.5 User Service

Handles:

- Requirement 13

Security Measures

- Encrypt personal data
 - Use authentication (JWT / OAuth)
 - Role-based access
-

4. Client Applications

Mobile + Web Apps

Support:

- Browse tickets
 - Select quantity
 - Identify user type
 - Purchase tickets
-

Example API Endpoints

```
GET /tickets
POST /orders
GET /orders/{id}
POST /payments
```



5. Error Handling

Invalid Orders (Req. 11)

Examples:

- Quantity = 0
- Missing ticket type
- Invalid user type

</> JSON



```
{  
  "error": "Invalid order",  
  "message": "Quantity must be greater than 0"  
}
```



6. Security & Privacy

Personal Data (Req. 13)

- Encrypt sensitive fields (AES)
- Use HTTPS everywhere
- Apply GDPR compliance principles

Payment Data (Req. 14)

- Tokenization
- No storage of:
 - CVV
 - Full card number



7. Reliability (Req. 12)

- Use **decimal types** (not float) for prices
- Centralize pricing logic in one service
- Add unit tests for pricing rules



8. Example Use Case

Scenario:

- Student buys 3 bundle tickets
- Base price: 10€

$$\text{Price} = 10 \times 0.8 \times 0.75 \times 3 = 18\text{€}$$


9. Suggested Tech Stack

- Backend: Node.js / Java Spring Boot / .NET
- Database: PostgreSQL
- Frontend: React (Web), Flutter/Swift/Kotlin (Mobile)
- Payment: Stripe / Adyen



10. Summary Mapping to Requirements

Requirement	Covered By
1–3	Ticket types
4–5	Mobile & Web apps
6–9	Pricing Service
10	Order Service
11	Validation logic
12	Pricing reliability
13	User Service security
14	Payment Service

If you want, I can:

- ↳ Turn this into UML diagrams
- ↳ Generate full backend code (Spring Boot / Node.js)
- Or ↳ design a database schema + SQL



Generate the full backend code using these requirements

📁 Ticketing System Backend (node

I've created a complete working backend in the canvas for you using **Node.js + Express** that implements all your requirements.

What you now have:

- ☒ Ticket types (single, period, bundle)
 - ☒ User types (student, senior, regular) with pricing adjustments
 - ☒ Pricing engine with modifiers
 - ☒ Order creation with validation
 - ☒ Support for multiple tickets per order
 - ☒ Error handling for invalid orders
 - ☒ Mock payment system with secure handling (no raw card storage)
 - ☒ Basic privacy via hashing/encryption
 - ☒ REST API endpoints for web & mobile apps
-

What you can do next

If you want to take this further, I can upgrade this into a **production-grade system**, for example:

- ↳ Add database integration (PostgreSQL + ORM)
- ↳ Implement authentication (JWT login/signup)
- ↳ Split into microservices (pricing, orders, payments)
- ↳ Add unit + integration tests
- ↳ Convert to TypeScript for scalability
- ↳ Build a Spring Boot / .NET version instead

Just tell me 👍



Add this requirement (The system shall allow users to create an account)

Ticketing System Backend (node · javascript)

Kopiera Redigera Ladda ner

```
// Ticketing System Backend (Node.js + Express)
// Updated: Adds account creation + authentication (Login)

const express = require('express');
const bodyParser = require('body-parser');
const crypto = require('crypto');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

const app = express();
app.use(bodyParser.json());

const JWT_SECRET = 'supersecret';

// In-memory DB (for demo purposes)
const users = [];
const tickets = [
  { id: 1, type: 'SINGLE', basePrice: 10 },
  { id: 2, type: 'PERIOD', basePrice: 50 },
  { id: 3, type: 'BUNDLE', basePrice: 80 }
];
const orders = [];

// Helpers
const USER_MODIFIER = {
  STUDENT: 0.8,
  SENIOR: 0.85,
  REGULAR: 1.0
};

const TICKET_MODIFIER = {
  SINGLE: 1.0,
  PERIOD: 0.9,
  BUNDLE: 0.75
};

function calculatePrice(userType, ticketType, basePrice, quantity) {
  if (quantity <= 0) {
    throw new Error('Quantity must be greater than 0');
  }

  const userMod = USER_MODIFIER[userType];
  const ticketMod = TICKET_MODIFIER[ticketType];

  if (!userMod || !ticketMod) {
    throw new Error('Invalid pricing configuration');
  }
}
```

```
    const price = basePrice * userMod * ticketMod * quantity;
    return Math.round(price * 100) / 100;
}

// Middleware for auth
function authenticate(req, res, next) {
    const token = req.headers['authorization'];
    if (!token) return res.status(401).json({ error: 'No token provided' });

    try {
        const decoded = jwt.verify(token, JWT_SECRET);
        req.user = decoded;
        next();
    } catch (err) {
        return res.status(401).json({ error: 'Invalid token' });
    }
}

// User Account Creation (NEW REQUIREMENT)
app.post('/auth/register', async (req, res) => {
    const { name, email, password, userType } = req.body;

    if (!name || !email || !password || !userType) {
        return res.status(400).json({ error: 'Missing fields' });
    }

    const existing = users.find(u => u.email === email);
    if (existing) {
        return res.status(400).json({ error: 'User already exists' });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    const user = {
        id: users.length + 1,
        name,
        email,
        password: hashedPassword,
        userType
    };

    users.push(user);

    res.json({ message: 'User registered successfully' });
});

// Login
app.post('/auth/login', async (req, res) => {
    const { email, password } = req.body;

    const user = users.find(u => u.email === email);
    if (!user) {
```

```
    return res.status(404).json({ error: 'User not found' });
  }

  const valid = await bcrypt.compare(password, user.password);
  if (!valid) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  const token = jwt.sign({ id: user.id, userType: user.userType }, JWT_SECRET);

  res.json({ token });
});

// Ticket Routes
app.get('/tickets', (req, res) => {
  res.json(tickets);
});

// Order Routes (protected)
app.post('/orders', authenticate, (req, res) => {
  try {
    const userId = req.user.id;
    const { items } = req.body;

    if (!items || items.length === 0) {
      return res.status(400).json({ error: 'Invalid order data' });
    }

    const user = users.find(u => u.id === userId);

    let totalPrice = 0;
    const processedItems = [];

    for (const item of items) {
      const ticket = tickets.find(t => t.id === item.ticketId);

      if (!ticket) {
        throw new Error('Invalid ticket');
      }

      const price = calculatePrice(
        user.userType,
        ticket.type,
        ticket.basePrice,
        item.quantity
      );

      totalPrice += price;

      processedItems.push({
        ticketId: ticket.id,
        quantity: item.quantity,
        price
      });
    }

    res.json({
      items: processedItems,
      totalPrice
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

```
    });  
  }  
  
  const order = {  
    id: orders.length + 1,  
    userId,  
    items: processedItems,  
    totalPrice,  
    status: 'CREATED'  
  };  
  
  orders.push(order);  
  res.json(order);  
  
} catch (err) {  
  res.status(400).json({ error: err.message });  
}  
});  
  
// Payment Route (mocked)  
app.post('/payments', authenticate, (req, res) => {  
  const { orderId, cardNumber } = req.body;  
  
  const order = orders.find(o => o.id === orderId);  
  if (!order) {  
    return res.status(404).json({ error: 'Order not found' });  
  }  
  
  const maskedCard = '****_****_****_' + cardNumber.slice(-4);  
  
  order.status = 'PAID';  
  order.payment = {  
    transactionId: crypto.randomUUID(),  
    maskedCard  
  };  
  
  res.json({ message: 'Payment successful', order });  
});  
  
// Start server  
app.listen(3000, () => {  
  console.log('Server running on port 3000');  
});
```

Done. I've updated your backend to support user account creation and authentication.

✅ What was added

- User registration endpoint: POST /auth/register

- **Login endpoint:** POST /auth/login
- **Password hashing** using bcrypt
- **JWT authentication**
- Protected routes (/orders , /payments)



How it works now

1. User creates an account (register)
2. User logs in → receives a token
3. Token is used to:
 - Buy tickets
 - Make payments

Example flow

 Bash



```
POST /auth/register
POST /auth/login → get token
POST /orders (with token)
POST /payments (with token)
```

If you want next steps, I can:

- Add **refresh tokens + logout**
- Add **email verification**
- Connect to a **real database**
- Or build a **frontend that uses this API**

